

Optimizing Java Programs Execution in an Auto grading environment

RnD Project Report
by

Penta Kusuma Kumari, 190050081

Sonnandh Kalaka, 190050055

Supervisor:

Prof. Varsha Apte



Department of Computer Science Engineering

Indian Institute of Technology Bombay

Mumbai 400076 (India)

7 December 2025

Acknowledgements

We want to thank my mentor Prof. Varsha Apte for helping us learn from our mistakes and for always attempting to guide us in the right direction. We value the lessons she taught us and want to always remember them. For their blessings, our parents and instructors have our sincere gratitude.

Penta Kusuma Kumari, 190050081

Sonnandh Kalaka, 190050055

IIT Bombay
7 December 2025

Abstract

In this report, we will discuss the main motivations for providing support for Java in existing e-learning platform Bodhitree's auto grader Evalpro 2022. We will discuss the need for finding an efficient execution plan for java programs and why executing similar to programs with existing language support is inefficient. Throughout the report we mention how our methodology i.e., our solution tackles issues in the naive execution of java programs as we go deep into our approach by walking through all the required tools and technologies and how we sew them together in an efficient solution which outperforms the existing naive solution by more than 50%. We shall conclude with discussions on the weaknesses in our approach and recommendations for future research work.

Table of Contents

Acknowledgements	i
Abstract	ii
Introduction	1
Analysis of Existing Solution	2
0.1 Executing java programs in current plan	2
0.1.1 Current Execution plan	2
0.1.2 Separate JVM for executing test-case	2
0.2 Major Drawbacks	2
0.2.1 Heavy overhead in spawning JVMs	2
Our Methodology	3
0.3 Technologies Used	3
0.3.1 Jshell	3
0.3.2 Unix pipes	5
0.4 Restriction of Execution time	6
0.4.1 /proc folder in linux	6
0.4.2 Calculating from stat file in /proc folder	6
0.5 Implementation	7
Analysis of our Methodology	9
0.6 Experiment values and graphs	9
0.6.1 Environment of the analysis	9
0.7 Details of the experiment performed	9
0.7.1 Identifying optimal number of jshells	9
0.7.2 Performance improvement using jshells	10
0.8 Python scripts for measurement	11
0.8.1 Naive Solution Implementation	11
0.8.2 Jshell creation and linking pipes	13
0.8.3 Running programs on jshells	14
Discussion	18
0.9 Weaknesses in our Methodology	18

0.9.1	Extensive I/O operations	18
0.9.2	Separate thread for polling	18
0.10	Difficulties in restricting memory	18
0.10.1	Definition of Memory in Linux	18
Conclusions		19
0.11	Summary of main findings	19
0.12	Recommendations for future research work	19

Introduction

IIT Bombay created the Bodhitree, an e-learning platform that now serves IIT Bombay students. It serves as the foundation for the instructor-student connection. Bodhitree can accommodate several courses on a single platform. The instructor uses it to host the course content. It includes a discussion area where students can easily post questions to be answered by the instructor. The instructor can also post the results of offline quizzes, and the student will only see his own grades, ensuring student grade privacy. It also has an emailing feature, which allows the instructor to send emails to anyone in his course.

Currently Evalpro 2022 is in development which is the successor of the previous generation Eval Pro 2020, whose main motivation is to fix previous issues while also creating an adaptable and maintainable code that would endure the passage of time. It is a Bodhitree component that provides an auto-evaluation system for programming languages such as C, Simplecpp, C++, Python. With the help of Evalpro, the instructor can create a programming assignment to his liking, as well as test cases and publish to the students as assignments which would be later assessed and graded by Evalpro.

Gradually one would want to add support for other languages such as Java, Kotlin and also upcoming new languages such as Rust and others. Intuitively one would want to provide support for the second most frequently used programming language Java. Though the current execution plan in Evalpro 2022 can support Java we will later see that this is ineffective and sub-optimal. This is inherently due to the fact that java programs require a virtual machine, the JVM (Java Virtual Machine) to execute java programs and for every test-case per program we run, this would require a new JVM to be spawned. This leads to many JVMs being spawned and killed and this leads to a lot of overhead and also not to mention that the compiler for java programs is written in Java, so compilation also requires a JVM.

So, Ideally the objectives of this research project are to design an efficient architecture and separate execution plan for java programs which ensures correctness and also improves performance. We explore ideas such as restriction on the number of active JVMs on the machine and possibilities to reuse an already alive JVM.

Moving forward, we shall refer to Our Solution as the methodology we follow to tackle issues in the existing architecture and execution plan which we will refer as Naive Solution or as Existing solution.

Analysis of Existing Solution

Here, we analyze the solution which can be implemented in Evalpro 2022 as per the architecture and execution plan currently available for supported programming languages. And also we go through some major drawbacks and inefficiencies in this method.

0.1 Executing java programs in current plan

0.1.1 Current Execution plan

The current execution plan involves three stages

- Compile
- Run
- Check

Compile stage is on a program i.e., every program is compiled once. Run stage is on a test-case level i.e., every test-case is coupled with a program and executed based on the programming language and the Check stage is also on a test-case level as the output of each test-case is checked with the expected output.

0.1.2 Separate JVM for executing test-case

that java programs require a virtual machine, the JVM (Java Virtual Machine) to execute java programs and also that the compiler for java programs is written in Java, so compilation also requires a JVM. For example, assuming if there are T test-cases per program, To grade a program one would have to spawn $T + 1$ JVMs to execute all the test-cases, where the T separate JVMs are from the test-cases and whereas the additional one JVM is required for the compilation at the start.

0.2 Major Drawbacks

0.2.1 Heavy overhead in spawning JVMs

We can clearly see in the above example that many JVMs are required to completely grade a program and this incurs heavy overheads both in terms of startup time and startup memory which can degrade the performance of the auto-grading server and increasing turnaround time of programs with different language.

Our Methodology

0.3 Technologies Used

0.3.1 Jshell

The Java Shell tool (JShell) is an interactive command-line tool introduced in Java 9. JShell is a Read-Evaluate-Print Loop tool (REPL) where programmers can directly code Java into the console, which is then compiled and executed on-the-fly. This means that we don't need to compile the java program before hand and can directly run it on Jshell. Jshell behaves like an infinite loop Java Program where a JVM is created for each one of them and doesn't create any extra new JVMs while executing the java code in it. As mentioned in 2.2 creating extra JVMs for each Java Program in the naively implemented solution is resource intensive and we can use this powerful lightweight java tool to not create new JVMs for each user program at each testcase. Jshell is also resistant to malicious code like `System.exit()`, it will throw an error without being terminated inappropriately.

In our solution, we will leverage the usecases of the Jshell by creating a jshell and running our user programs sequentially.

The steps to open, run and execute a user program for a single testcase:

- **Open Program**

User program are first loaded into the jshell i.e. the command open file will read the file and stores the content into jshell memory.

Listing 1: loading the java program

```
jshell> /open A.java
```

- **Set Input-Output**

Initially, we were setting the I/O using the following commands, but this step is taking more time. To optimise, we were taking input directly from the UNIX-pipe (which will be explained in future) just like commands and output is generated in the command line which will be retrieved and written into a file.

But in order to do so i.e., write to the pipe and input to be taken from the pipe itself and not via above mentioned methods. We face a subtle issue of closing the input stream i.e., general programs written in java take input from the Scanner class and

after taking the input they generally close the scanner and this leads to input stream closing and subsequent programs which run in the jshell won't be able to use the `InputStream` as it is closed by the program.

In order to overcome this, we have to wrap `InputStream` with a decorator so that it can give additional features as to not close whenever a close function is called, we simply ignore the close function in the wrapper. And through this small modifications which can be added into the jshell via a simple startup script.

Listing 2: Setting I/O for programs

```
jshell> System.setIn(new FileInputStream(\"input.txt\"))
)
jshell> System.setOut(new PrintStream(new
    FileOutputStream(\"output.txt\")))
```

Listing 3: Wrapper around `System.in`

```
public class UnclosableDecorator extends InputStream {

    private final InputStream inputStream;

    public UnclosableDecorator(InputStream inputStream)
    {
        this.inputStream = inputStream;
    }

    @Override
    public void close() throws IOException {
        //do nothing
    }
}
```

- **Execute**

This is the step that will call the main function and executes the given java program and writes the results to the log file assigned to the jshell. Since, our system is asynchronous we have to wait for our output to be generated and as per design of Bodhi tree we know that the frontend constantly checks the status of the output in the new Eval Pro 2022. And hence whenever the frontend sends a request to

check whether an output has been generated or not, we will check the log file for the corresponding input in the request.

Listing 4: Executing the main func of Java program

```
jshell> A.main(new String[]{})
```

- **Reset**

We will reset after every user program to maintain the isolation between the programs. This step will erase state and all the memory of the Jshell by creating a new remote execution control (this is the part of jshell where execution is done). This step is a little bit costlier in terms of time, so we will be doing reset after running all the testcases of the user program.

Listing 5: resetting the Jshell

```
jshell> /reset
```

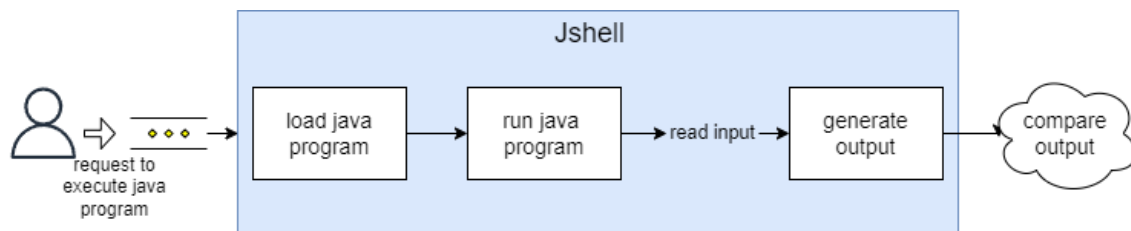


Figure 1: Flow of running a Java Program in Jshell

0.3.2 Unix pipes

Unix Pipe is named pipe, a mechanism used for inter-process communication that allows two or more processes to communicate with each other by sending and receiving data through a file-like interface. In our solution, we will be using pipes to handle user requests asynchronously by converting the requests into set of commands to be executed and writing them into the pipe which are eventually read and executed by Jshell. Each jshell is associated with a unix pipe and the commands that are written into the pipe are executed by this associated Jshell. The file `create_pipes.sh` is responsible for creating the pipes and associating them with a jshell and the file `delete_pipes.sh` is responsible to kill the created Jshell processes and delete the pipes.

Code in `create_pipes.sh` which creates pipes:

```
# creates pipes
for i in $(seq 1 $NUM_PIPES)
do
    mkfifo $DIR/pipe$i
done
```

This command below creates a background Jshell process and directs the end of pipe to the Jshell. The interface Output is written into a log file which can be for future purposes like retrieving the output of a java program etc.,

```
tail -f $DIR/pipe$i | jshell >> $DIR/jshell_log$i.txt &
```

The Process ID's of the above created Jshell processes will be written into shell_pids.txt. The ID's added will be used in delete_pipes.sh to send kill signal to the processes.

0.4 Restriction of Execution time

We plan to limit cpu time for each program run on the jshell via polling, which is also used in traditional sandbox environments such as safeexec. We make sure that our thread always has to be running whenever our jshell is running. Instead of making it exactly running when the other is running we can assign this polling process to run on a separate single core and make sure that this is the only job running on this core.

0.4.1 /proc folder in linux

The /proc folder in Linux is a virtual file system that provides a way for users and applications to access information about the system and its processes in real-time. The /proc file system is not a physical file system that exists on disk, but rather a mechanism for exposing information about the state of the system and its processes to user-space applications and utilities. We plan to leverage this folder to polling this rather than the process itself which can be beneficial as we have to read only a set of files.

0.4.2 Calculating from stat file in /proc folder

The stat file in the /proc directory is a virtual file that provides information about the status of a process. It contains a single line of text with various fields separated by spaces, each of which provides information about a different aspect of the process.

We can parse this file and read the appropriate columns i.e., system cpu time and user cpu time and add the both to calculate the total cpu time allotted to a process. And this information can restrict cpu time usage of the process running inside jshell.

0.5 Implementation

The request to execute at the user level is a Java Program with a set of testcases present in Input directory and we will be receiving many requests like this to execute. In our solution, we will be creating a fixed number of Jshells with its pipes as mentioned in 3.1.2. To handle these requests we will be creating a set of commands for a request and direct them to the Jshells using round robin algorithm. The logic for this is implemented in run_prog_level.sh file.

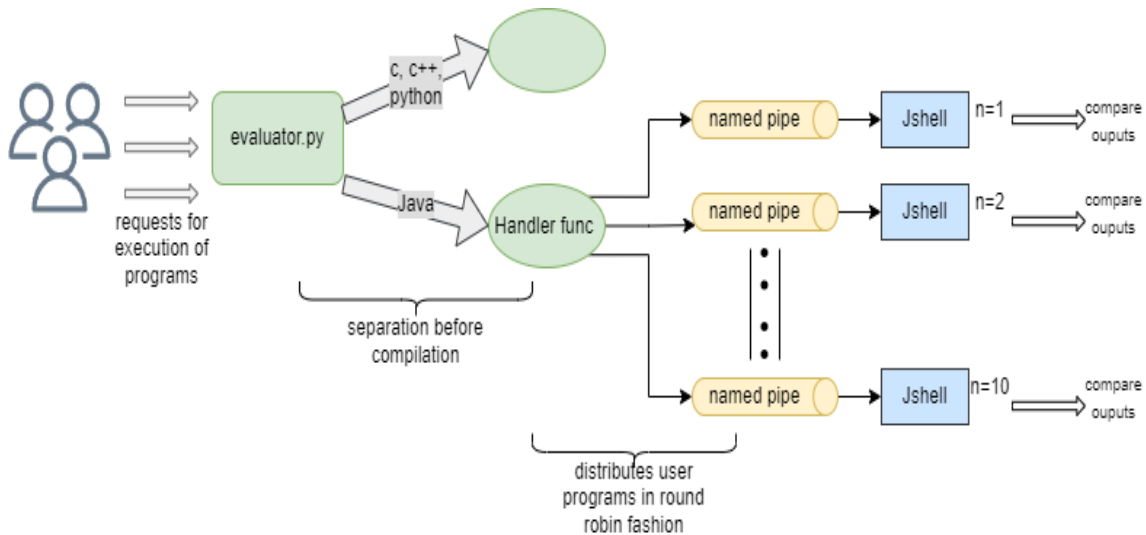


Figure 2: Architecture of java program execution

The execution of java programs are branched out at compile stage in `evaluator.py` in Bodhitree and from here on we will have a separate path for execution of Java programs. The directory setup is a step taken before branching out where a directory with the program to run and set of testcases were created. We don't require any compilation for java programs since we can directly load and run in jshell.

The handler func for the java programs creates the set of commands to execute in jshell and write them into one of the pipe using round robin algorithm. We write to one end of the pipe and other end of the pipe is fed into a jshell. Every pipe resembles a unique jshell. The jshell associated with this pipe reads and executes the commands one by one and the output will be written into the log file of that jshell. This is where we are mimicking the celery worker threads and handling the requests asynchronously.

And similar to existing design where the frontend sends request to the backend to check the status of the program execution. We plan to handle it similarly, when a status update request for an program comes to the backend, we simply check whether the output has been written to the corresponding log file. If written we extract the output from the logfile and if not we continue the same again.

Analysis of our Methodology

0.6 Experiment values and graphs

0.6.1 Environment of the analysis

The analysis mentioned below are performed on a QEMU assisted Linux (Ubuntu) virtual machine with ARM64 architecture which has been allotted 8GB of physical RAM and three cores running on top of an Mac Book Pro with 512GB SSD, 16 GB RAM and 8 cores.

0.7 Details of the experiment performed

The objective of the experiment is to analyze and compare performance of two methods to execute java programs mentioned above. In this experiment we calculate the total time taken to execute a set of programs in varying conditions such as number of programs to execute or in the restriction in the number of JVMs.

The program that was incorporated into the experiment and in fact all the programs were a simple addition of numbers from 1 to n where n is specified by the user taken as input.

0.7.1 Identifying optimal number of jshells

In order to find the optimal number of jshells we consider an experiment where we have 100 user programs each with 20 test cases per program that are required to be executed. Keeping the number of programs constant, we vary the number of jshells that can be up at any instant and we execute all the 100 programs and measure the total time required and plot them. We find that the values of total time taken when number of jshells are to be 10, 12, 15 and 20 to be nearly exact and not much in difference.

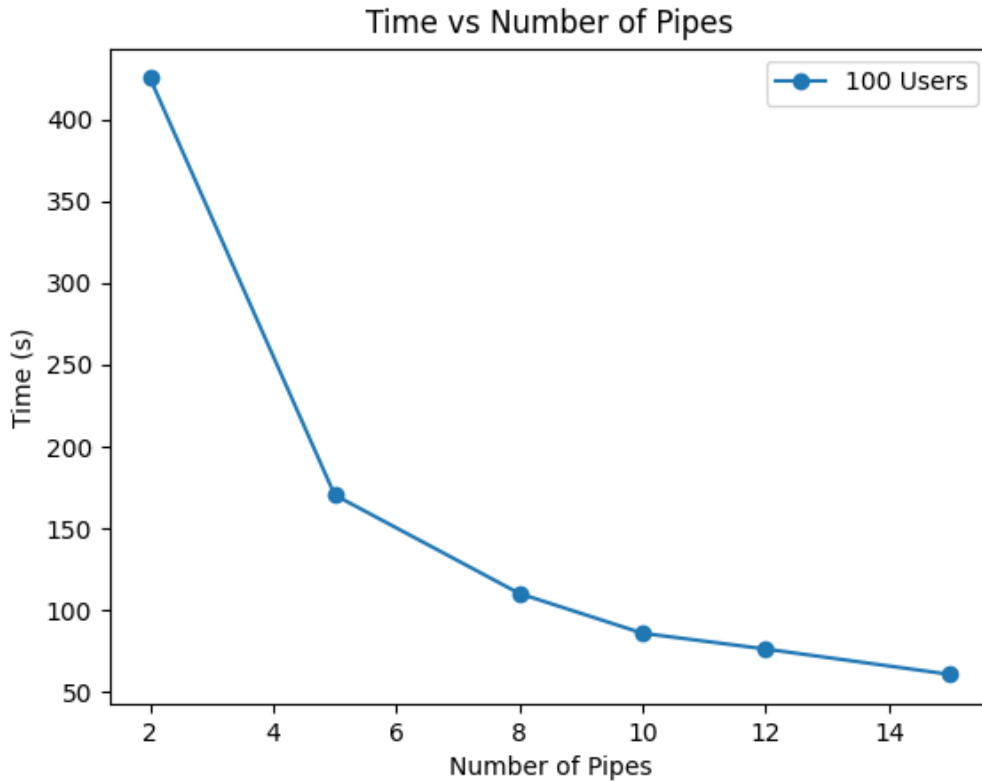


Figure 3: Time vs Number of jshells

And for values less than 10 we find that there is steady decrease in the total time. Although our findings are very much dependant on the machine itself and the optimal value must be calculated for every different machine. We firmly believe that having 10 jshells running in background on the autograding machine is a good choice for all machines which have atleast 8GB RAM.

0.7.2 Peformance improvement using jshells

To compare the two strategies that we have, firstly to ensure fairness we restrict the number of JVMs/jshells that can be active at any instant to 10 i.e., the number of JVMs that can be spawned by the naive solution is 10 and the number of jshells running on the machine is 10.

Given these conditions, we vary the number of programs that are needed to be executed starting out with 1 program to 100 programs.

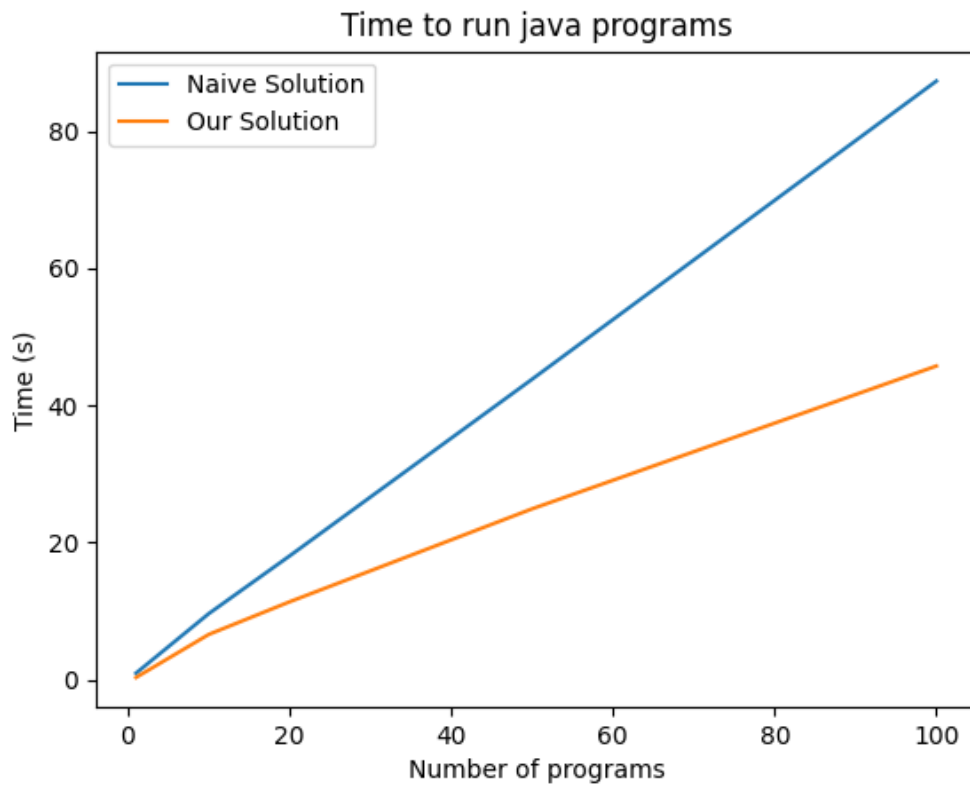


Figure 4: Time vs Number of programs

Results show us that there is substantial improvement in the total time taken with effective improvement percentage to be more than 50% which means that the total time has been reduced to half the time taken by the naive solution.

0.8 Python scripts for measurement

0.8.1 Naive Solution Implementation

Listing 6: Implementation of Naive Solution

```
#!/bin/bash

# exit if arguments are not provided
if [ $# -ne 1 ]
then
    echo "Usage: _$0_<NUM_TESTS>"
    exit 1
fi
```



```
NUM_TESTS=$1

# Set the maximum number of programs to run at the same
  time
MAX_PROGRAMS=10

PWD=$(pwd)

# all directories in the current directory whose prefix is
  "test"

# loop through all directories in the current directory
for dir in user*; do

  # set -cp
  cp="$dir"

  # Get absolute path of the directory
  dir="$PWD/$dir"
  # Get the program name
  program="$dir/Sum.java"

  # Get the input and output directories
  input_dir="$dir/input"

  output_dir="$dir/output"

  # wait if the maximum number of programs are running
  while [ $(jobs | wc -l) -ge $MAX_PROGRAMS ]; do
    wait -n
  done

  # Compile the program
  javac "$program"
```

```

# Run the program NUM_TESTS times in the background
for i in $(seq 1 $NUM_TESTS); do

    # Run the program
    (java -cp "$cp" Sum < "$input_dir/file$i.txt" > "
        $output_dir/out$i.txt") &

    while [ $(jobs | wc -l) -ge $MAX_PROGRAMS ]; do
        wait -n
    done

done

# Wait if the maximum number of programs are running
while [ $(jobs | wc -l) -ge $MAX_PROGRAMS ]; do
    wait -n
done

done
wait

```

0.8.2 Jshell creation and linking pipes

Listing 7: Creation of pipes and jshells

```

#!/bin/bash

# stop if number of pipes is not specified
if [ $# -eq 0 ]
then
    echo "Number_of_pipes_not_specified"
    exit 1
fi

PWD='pwd'
DIR=$PWD/pipes

# create pipes directory if not exists

```

```
mkdir -p $DIR

# clear pipes directory
rm -f $DIR/*

NUM_PIPES=$1

# create pipes
for i in $(seq 1 $NUM_PIPES)
do
    # mkfifo command: creates a FIFO special file with name
    # pipe$i
    mkfifo $DIR/pipe$i
done

# create shell_pids.txt file: contains pids of all shell
# processes
touch $PWD/shell_pids.txt

# create jshell processes
for i in $(seq 1 $NUM_PIPES)
do

    # pipe output from pipe$i to jshell process
    # create jshell_log$i.txt file: contains output of
    # jshell process
    tail -f $DIR/pipe$i | jshell >> $DIR/jshell_log$i.txt &

    # store pid of jshell process in shell_pids.txt file
    echo $! >> $PWD/shell_pids.txt

done
```

0.8.3 Running programs on jshells

Listing 8: Running programs on jshell

```
#!/bin/bash
```

```
# take number of programs and no of pipes as input
if [ $# -ne 2 ]
then
    echo "Number_of_programs_or_number_of_pipes_not_
        specified"
    exit 1
fi

# num of files to execute
NUM_PROGRAMS=$1

# num of pipes to use
NUM_PIPES=$2

echo "Running_$NUM_PROGRAMS_programs_using_$NUM_PIPES_pipes
"

PWD='pwd'

# print time in nanoseconds at start in all pipes
for i in $(seq 1 $NUM_PIPES)
do
    # set start script in jshell
    echo "/set_start_startup.jsh" > $PWD/pipes/pipe$i

    # reset jshell to activate start script
    echo "/reset" > $PWD/pipes/pipe$i

    # start timer ( used for measuring total time taken by
        all programs )
    echo "System.nanoTime()" > $PWD/pipes/pipe$i
done
```

```
# assign programs to jshell in a round robin fashion
for i in $(seq 1 $1)
do
    # get input and output directories, pipes directory and
    # files directory
    FILES_DIR=$PWD/programs/user$i
    INPUT_DIR=$FILES_DIR/input
    OUTPUT_DIR=$FILES_DIR/output
    PIPES_DIR=$PWD/pipes

    # get java program name present in FILES_DIR
    JAVA_FILE='ls $FILES_DIR | grep .java'

    echo "Running_$JAVA_FILE"

    # get pipe number
    pipe_num=$((i % NUM_PIPES))

    # if pipe_num is 0, assign it to last pipe
    if [ $pipe_num -eq 0 ]
    then
        pipe_num=$NUM_PIPES
    fi

    # set pipe number and java file name
    JAVA_FILE=$PWD/programs/user$i/$JAVA_FILE
    PIPE=$PIPES_DIR/pipe$pipe_num

    # open java file in jshell
    echo "/open_$JAVA_FILE" > $PIPE

    CLASS_NAME='echo $JAVA_FILE | rev | cut -d "/" -f 1 |
        rev | cut -d "." -f 1'

    #for each 1to10.txt file in input directory
```

```
for j in $(seq 1 20)
do
    # run testcase
    echo "$CLASS_NAME.main(new_String[]{})" > $PIPE

    # trim end of line from input file
    sed -i -e 's/[[:space:]]*$//' $INPUT_DIR/file$j.txt

    # print input to jshell
    while read line
    do
        echo "$line" > $PIPE
    done < "$INPUT_DIR/file$j.txt"

done

# can uncomment to time every program
# echo "System.nanoTime()" > $PIPE

# reset jshell ( remove all variables, methods, classes
# , etc.)
# frequency of reset can be changed
echo "/reset" > $PIPE

done

# To measure total time taken by all programs
# print time in nanoseconds at end in all pipes
for i in $(seq 1 $NUM_PIPES)
do
    echo "System.nanoTime()" >> $PWD/pipes/pipe$i
done
```

Discussion

0.9 Weaknesses in our Methodology

0.9.1 Extensive I/O operations

We know that file writing is a time consuming process and extensive writing can lead to performance degradation. Commands and also the input are written on an layer above the jshell i.e., the pipe and this is again fed back to the program via I/O. This additional writing to the above layer incurs loss in time and we have to minimize the number of operations that have to be written into the pipe. Instead of writing multiple commands to the pipe itself, it can be better to write commands to a jshell script and load that file into the jshell and execute commands one by one.

0.9.2 Separate thread for polling

As discussed earlier we have to setup an additional thread to poll cpu time of each process being run in the jshell. This management of a process is difficult and complex as there can be issues such as synchronization between the polling thread and the jshell process and these can lead to inaccurate results.

0.10 Difficulties in restricting memory

0.10.1 Definition of Memory in Linux

Using in-built linux tools to poll memory can be a challenging situation because of the ambiguous definition of memory in linux systems. We have to first define what exactly memory of a process do we mean, is the RSS (Resident Set Size) or USS (Unique Set Size) or VSS (Virtual Set Size) or PSS (Proportional Set Size) or is it some complex combination of the above mentioned types. And also complications such as different tools offering different method of calculating the above terms leads to ambiguity in memory calculations.

Conclusions

0.11 Summary of main findings

The research project aims to design an efficient architecture and separate execution plan for Java programs that ensures correctness while improving performance. The Existing Solution is analyzed, which consists of three stages: Compile, Run, and Check. The drawback of the Existing Solution is that it incurs heavy overheads in spawning JVMs, degrading performance and increasing turnaround time. The proposed methodology uses Jshell, a lightweight Java tool, to create a Jshell and run user programs sequentially. Unix pipes are used to handle user requests asynchronously by converting the requests into sets of commands to be executed and written into the pipe. Finally, each Jshell is associated with a Unix pipe, and the commands written into the pipe are executed by the associated Jshell.

An experiment was performed on a QEMU assisted Linux virtual machine with ARM64 architecture. The objective was to identify the optimal number of jshells and to measure the performance improvement using jshells. The results showed that having 10 jshells running in the background on the autograding machine is a good choice for all machines which have at least 8GB RAM. There was substantial improvement in the total time taken with an effective improvement percentage of more than 50

0.12 Recommendations for future research work

There are many possible ways in which this research work can be extended. Some of them being are listed as follows:

- Build a wrapper to abstract all the details, which can later be named as a special tool for performance oriented execution of java programs
- Solve problem of exact memory usage of a process inside jshell instead of restricting for entire jshell itself